Emergency Training Simulator POC

Meeting deck / backend-first local proof

POC

Backend-first foundation for a future adaptive training simulator

5-minute update + 5-minute demo

The short version

Meeting deck / backend-first local proof

POC

I am proving the foundation before building the spectacle.

- trainee action in
- backend-owned state change
- rule check
- client update
- log, replay, review

Why this project exists

Meeting deck / backend-first local proof

POC

Good training needs more than a scene.

- repeatable scenarios
- visible consequences
- environmental cues
- instructor review
- a clear record of what happened

The trap I am avoiding

POC

The shiny parts can wait.

- Unreal
- generated 3D assets
- sensors
- physics
- real AI calls

First question: can the system keep the truth straight?

The core loop

Meeting deck / backend-first local proof

POC

```
trainee action
-> backend
-> scenario state
-> validation
-> client update
-> JSONL log
-> replay / review
```

This is the spine of the project.

What works now

Meeting deck / backend-first local proof

POC

The local POC can already run end-to-end.

- FastAPI WebSocket backend
- deterministic YAML scenario engine
- POC validation rules
- JSONL session logs
- replay and review summaries
- one-command demo

Current demo scenario

Meeting deck / backend-first local proof

POC

A tiny smoke-room software scenario.

- wooden chair may block the path
- entry door starts closed
- trainee actions change backend state
- warnings appear when cues are ignored

Not field guidance. Just a POC scenario.

Why the chair and door matter

Meeting deck / backend-first local proof

POC

The room should not be ignored.

- obstacles can block progress
- doors can change access
- future objects can carry metadata
- physics stays future work

Right now: state and rule checking only.

The strongest proof point

POC

The demo does not just run.

It explains what happened.

- actions taken
- final state
- warnings and errors
- replay verification
- environment cue status

Trust boundary

Meeting deck / backend-first local proof

POC

The client requests actions.

The backend decides truth.

- state changes
- validation results
- final outcome
- session log
- review summary

Security baseline

Meeting deck / backend-first local proof

POC

This is POC hardening, not production security.

Current guardrails reject:

- strange IDs
- unsafe message shapes
- oversized payloads
- wrong sequence numbers
- duplicate event IDs

Future Unreal role

Meeting deck / backend-first local proof

POC

Unreal should display state, not own scenario logic.

- connect over WebSocket
- send simple trainee actions
- receive backend state
- show text/debug state first
- scenes and gameplay later

Submission packet vs meeting deck

Meeting deck / backend-first local proof

POC

The repo now separates the deliverables.

- packet: submit/read
- deck: talk through live
- demo: prove the loop
- docs: explain the system

No more PDF confetti.

Next useful steps

POC

Keep adding one careful layer at a time.

1. tighten display contract
2. improve review output
3. enrich object metadata
4. test tiny Unreal display
5. review assets in Blender
6. add physics later

Demo plan

Meeting deck / backend-first local proof

POC

Run:

```
python scripts/run_local_demo.py
```

Show:

- happy path
- warning/error path
- replay verification
- review summary

Closing

Meeting deck / backend-first local proof

POC

The project is moving in the right order.

First prove the loop.

Then make it visible.

Then make it immersive.

One monster at a time.